

torch.lobpcg#

<https://pytorch.org/docs/stable/generated/torch.lobpcg.html#torch.lobpcg>

Description:

Find the k largest (or smallest) eigenvalues and the corresponding eigenvectors of a symmetric positive definite generalized eigenvalue problem using matrix-free LOBPCG methods. This function is a front-end to the following LOBPCG algorithms selectable via method argument: method="basic" - the LOBPCG method introduced by Andrew Knyazev, see [Knyazev2001]. A less robust method, may fail when Cholesky is applied to singular input. method="ortho" - the LOBPCG method with orthogonal basis selection [StathopoulosEtal2002]. A robust method. Supported inputs are dense, sparse, and batches of dense matrices.

Function Signature

```
torch.lobpcg(A, k=None, B=None, X=None, n=None, iK=None,
niter=None, tol=None, largest=None, method=None,
tracker=None, ortho_iparams=None, ortho_fparams=None,
ortho_bparams=None)[source]#
```

Parameters

Returns

Return type

Returns:

E (Tensor)

Notes & Warnings

NOTE

Note In general, the basic method spends least time per iteration. However, the robust methods converge much faster and are more stable. So, the usage of the basic method is generally not recommended but there exist cases where the usage of the basic method may be preferred.

⚠ WARNING

Warning The backward method does not support sparse and complex inputs. It works only when B is not provided (i.e. `B == None`). We are actively working on extensions, and the details of the algorithms are going to be published promptly.

⚠ WARNING

Warning While it is assumed that A is symmetric, A.grad is not. To make sure that A.grad is symmetric, so that $A - t * A.grad$ is symmetric in first-order optimization routines, prior to running lobpcg we do the following symmetrization map: $A \rightarrow (A + A.t()) / 2$. The map is performed only when the A requires gradients.

⚠ WARNING

Warning LOBPCG algorithm is not applicable when the number of A's rows is smaller than 3x the number of requested eigenpairs n.

Detailed Documentation

Documentation

Find the k largest (or smallest) eigenvalues and the corresponding eigenvectors of a symmetric positive definite generalized eigenvalue problem using matrix-free LOBPCG methods.

This function is a front-end to the following LOBPCG algorithms selectable via method argument:

method="basic" - the LOBPCG method introduced by Andrew Knyazev, see [Knyazev2001]. A less robust method, may fail when Cholesky is applied to singular input.

method="ortho" - the LOBPCG method with orthogonal basis selection [StathopoulosEtal2002]. A robust method.

Supported inputs are dense, sparse, and batches of dense matrices.

In general, the basic method spends least time per iteration. However, the robust methods converge much faster and are more stable. So, the usage of the basic method is generally not recommended but there exist cases where the usage of the basic method may be preferred.

The backward method does not support sparse and complex inputs. It works only when B is not provided (i.e. $B == \text{None}$). We are actively working on extensions, and the details of the algorithms are going to be published promptly.

While it is assumed that A is symmetric, $A.\text{grad}$ is not. To make sure that $A.\text{grad}$ is symmetric, so that $A - t * A.\text{grad}$ is symmetric in first-order optimization routines, prior to running `lobpcg` we do the following symmetrization map: $A \rightarrow (A + A.t()) / 2$. The map is performed only when the A requires gradients.

LOBPCG algorithm is not applicable when the number of A's rows is smaller than 3x the number of requested eigenpairs n.

A (Tensor) – the input tensor of size $(*,m,m)(*,m,m)(*,m,m)$

k (integer, optional) – the number of requested eigenpairs. Default is the number of XXX columns (when specified) or 1.

B (Tensor, optional) – the input tensor of size $(*,m,m)(*,m,m)(*,m,m)$. When not specified, B is interpreted as identity matrix.

X (tensor, optional) – the input tensor of size $(*,m,n)(*,m,n)(*,m,n)$ where $k \leq n \leq m$. When specified, it is used as initial approximation of eigenvectors. X must be a dense tensor.

n (integer, optional) – if XXX is not specified then n specifies the size of the generated random approximation of eigenvectors. Default value for n is k. If XXX is specified, any provided value of n is ignored and n is automatically set to the number of columns in XXX.

iK (tensor, optional) – the input tensor of size $(*,m,m)(*,m,m)(*,m,m)$. When specified, it will be used as preconditioner.

niter (int, optional) – maximum number of iterations. When reached, the iteration process is hard-stopped and the current approximation of eigenpairs is returned. For infinite iteration but until convergence criteria is met, use -1.

tol (float, optional) – residual tolerance for stopping criterion. Default is $\text{feps}^{**} 0.5$ where feps is smallest non-zero floating-point number of the given input tensor A data type.

largest (bool, optional) – when True, solve the eigenproblem for the largest eigenvalues. Otherwise, solve the eigenproblem for smallest eigenvalues. Default is True.

method (str, optional) – select LOBPCG method. See the description of the function above. Default is “ortho”.

tracker (callable, optional) –

a function for tracing the iteration process. When specified, it is called at each iteration step with LOBPCG instance as an argument. The LOBPCG instance holds the full state of the iteration process in the following attributes:

iparams, fparams, bparams - dictionaries of integer, float, and boolean valued input parameters, respectively

ivars, fvars, bvars, tvars - dictionaries of integer, float, boolean, and Tensor valued iteration variables, respectively.

A, B, iK - input Tensor arguments.

E, X, S, R - iteration Tensor variables.

ivars[“istep”] - the current iteration step X - the current approximation of eigenvectors E - the current approximation of eigenvalues R - the current residual
ivars[“converged_count”] - the current number of converged eigenpairs tvars[“rerr”] - the current state of convergence criteria

Note that when tracker stores Tensor objects from the LOBPCG instance, it must make copies of these.

If tracker sets bvars[“force_stop”] = True, the iteration process will be hard-stopped.

ortho_iparams (dict, optional) – various parameters to LOBPCG algorithm when using method=“ortho”.

ortho_fparams (dict, optional) – various parameters to LOBPCG algorithm when using

method="ortho".

ortho_bparams (dict, optional) – various parameters to LOBPCG algorithm when using method="ortho".

tensor of eigenvalues of size $(*,k)(*,k)(*,k)$

X (Tensor): tensor of eigenvectors of size $(*,m,k)(*,m,k)(*,m,k)$

[Knyazev2001] Andrew V. Knyazev. (2001) Toward the Optimal Preconditioned Eigensolver: Locally Optimal Block Preconditioned Conjugate Gradient Method. SIAM J. Sci. Comput., 23(2), 517-541. (25 pages)
<https://epubs.siam.org/doi/abs/10.1137/S1064827500366124>

[StathopoulosEtal2002] Andreas Stathopoulos and Kesheng Wu. (2002) A Block Orthogonalization Procedure with Constant Synchronization Requirements. SIAM J. Sci. Comput., 23(6), 2165-2182. (18 pages)
<https://epubs.siam.org/doi/10.1137/S1064827500370883>

[DuerschEtal2018] Jed A. Duersch, Meiyue Shao, Chao Yang, Ming Gu. (2018) A Robust and Efficient Implementation of LOBPCG. SIAM J. Sci. Comput., 40(5), C655-C676. (22 pages) <https://arxiv.org/abs/1704.07458>